



Informe de desarrollo del trabajo práctico Subasta ya

integrantes

Nestor zurita vergara.

Pone tu nombre completo soffiiiiiiiiiii..

Materia

Proyecto de software.

Docente a cargo:

Comision : 3, sab.(08:00-12:00)

Indice:

1. Introduccion:.....	1
2. Actores del sistema.....	
3. Requerimientos funcionales y casos de uso.....	
4. Requerimientos no funcionales	
5. Modelo de domino.....	
7. Diagram de clases.....	
8. Arquitectura del sistema.....	
9. Desarrollo de backend.....	
10. Desarrollo de frontend.....	
11. Base de datos y migraciones.....	
12. Pruebas y validacion.....	
13. conclusion.....	
14. bibliografia.....	

Introduccion:

En el siguiente informe se presentará la resolución de la problemática dada por el profesor de proyecto de software, que consta de la siguiente consigna.

"SubastaYa" es una plataforma web de subastas en tiempo real y comercio electrónico que busca modernizar las compras y ventas competitivas en línea.

El modelo de negocio se apoya en dos pilares críticos:

1. *Confianza y Solvencia Económica*: En las plataformas tradicionales, los usuarios suelen ofertar sin tener fondos reales, generando transacciones canceladas y fricción con los vendedores. En SubastaYa, toda puja debe estar respaldada por saldo real congelado en garantía (Escrow) dentro de la billetera virtual del usuario.

2. *Juego Limpio (Fair Play en Subastas)*: Muchos sistemas sufren de sniping (bots o usuarios que ofertan en el último milisegundo para ganar sin dar tiempo a respuesta). Para evitarlo, la plataforma incorpora un mecanismo de extensión dinámica de tiempo (Anti-sniping) y una interfaz altamente reactiva donde el usuario tiene visibilidad total del estado de las ofertas en vivo.

El objetivo del equipo de desarrollo es construir una solución web completa (Frontend, Backend y Base de Datos) que ofrezca una experiencia visual atractiva y fluida para compradores y vendedores, respaldada por un motor transaccional seguro y confiable.

Según esta consigna debemos diseñar y planificar todo el procedimiento de las resolución desde cero.

Actores del sistema.

Como primer paso en la resolución de esta problemática empezaremos con la identificación de los actores que interactúan con el sistema. Tales como **usuario** tanto como vendedor o comprador, **administrador y sistema de pagos**.

Dichos actores son las entidades que hacen que el sistema tenga sentido y que funcione correctamente, a continuación se detallan sus roles y sus responsabilidades:

Usuario

Ya sea comprador o vendedor son las personas que participan en la compra y venta de las subastas.

Comprador: realiza pujas por los productos.

- **Interacciones principales.** Iniciar sesión, consultar subastas, gestionar su billetera virtual y realizar ofertas.

Vendedor: Es la persona que publica los productos en la subasta.

- **Interacciones principales.** Crea las subastas además de definir los precios, recibe los pagos al finalizar la subasta.

Administrador.

Es el encargado de que el sistema esté en un buen funcionamiento, darle mantenimiento y resolver los conflictos.

- **Interacciones principales.** Revisa las auditorías, verifica la integridad de transacciones y gestiona los usuarios.

Sistema de pagos o proceso automatico(worker)

Es el encargado de gestionar los fondos y pagos durante la subasta.

- **Interacciones principales.** Automatizar los pagos, liberar fondos al vendedor y registrar cada movimiento del dinero dentro del sistema.

Requerimientos funcionales y sus casos de uso

Definición de requerimientos funcionales(RF): son especificaciones que describen las acciones, procesos y respuestas que el sistema debe realizar ante determinadas entradas o situaciones. Indican qué servicios ofrece el sistema, cómo debe comportarse y qué restricciones aplican a esas funciones.

En otras palabras, establecen las funcionalidades esperadas desde el punto de vista del usuario y del entorno operativo.

En este informe se separaran los RF por gestión:

Gestión de Usuarios

RF-01: El sistema deberá permitir el registro de nuevos usuarios mediante un formulario de alta.

RF-02: El sistema deberá autenticar a los usuarios mediante credenciales válidas para acceder a la plataforma.

RF-03: El sistema deberá permitir a cada usuario consultar y actualizar su información personal.

RF-04: El sistema deberá administrar los distintos roles de usuario (comprador, vendedor y administrador).

Gestión de Catálogo y Exploración de Subastas

RF-05: El sistema deberá permitir explorar el catálogo de subastas disponibles.

RF-06: El sistema deberá filtrar las subastas por estado (Activas, Próximas y Finalizadas).

RF-07: El sistema deberá filtrar las subastas por categoría.

RF-08: El sistema deberá filtrar las subastas por rango de precios.

RF-09: El sistema deberá ordenar las subastas por menor tiempo restante o mayor oferta actual.

RF-10: El sistema deberá mostrar en cada publicación la imagen, título, categoría, oferta actual, cantidad de ofertas y contador regresivo.

Gestión de Creación y Publicación de Subastas

RF-11: El sistema deberá permitir al vendedor publicar productos destinados a subasta.

RF-12: El sistema deberá registrar la información del producto, incluyendo título, descripción, imágenes y categoría.

RF-13: El sistema deberá permitir definir el precio base, el incremento mínimo y la duración de la subasta.

RF-14: El sistema deberá permitir configurar la fecha y hora de inicio y finalización de la subasta.

RF-15: El sistema deberá validar que la fecha de finalización sea posterior a la fecha de inicio.

RF-16: El sistema deberá validar que el precio base y el incremento mínimo sean valores positivos.

RF-17: El sistema deberá permitir al vendedor cancelar una publicación únicamente cuando aún no existan ofertas registradas.

Gestión de Sala de Subasta en Vivo

RF-18: El sistema deberá iniciar automáticamente una subasta en la fecha y hora configuradas.

RF-19: El sistema deberá mostrar en tiempo real el precio actual y la oferta líder.

RF-20: El sistema deberá mostrar el historial cronológico de las ofertas realizadas.

RF-21: El sistema deberá registrar cada oferta conservando su fecha, hora y usuario oferente.

RF-22: El sistema deberá rechazar ofertas cuyo importe sea inferior o igual a la oferta vigente.

RF-23: El sistema deberá impedir que un usuario oferte sin saldo suficiente en su billetera virtual.

RF-24: El sistema deberá sugerir automáticamente el próximo valor de puja utilizando el incremento mínimo.

RF-25: El sistema deberá permitir ingresar un monto personalizado superior al valor mínimo sugerido.

RF-26: El sistema deberá indicar visualmente al usuario si lidera la subasta o si fue superado por otra oferta.

RF-27: El sistema deberá notificar la confirmación de una puja, fondos insuficientes y la extensión del tiempo de la subasta.

RF-28: El sistema deberá detectar ofertas realizadas dentro del período de protección anti-sniping.

RF-29: El sistema deberá extender automáticamente el tiempo restante de la subasta cuando corresponda.

Gestión de Billetera Virtual y Fondos

RF-30: El sistema deberá permitir a los usuarios acreditar saldo en su billetera virtual mediante un formulario de pago.

RF-31: El sistema deberá consultar el saldo disponible antes de autorizar cualquier oferta.

RF-32: El sistema deberá mostrar el saldo total, el saldo retenido y el saldo disponible del usuario.

RF-33: El sistema deberá congelar automáticamente el importe ofertado como garantía (escrow) al registrar una puja válida.

RF-34: El sistema deberá liberar los fondos retenidos cuando el usuario sea superado por otra oferta.

RF-35: El sistema deberá transferir el importe retenido al vendedor cuando la subasta finalice exitosamente.

RF-36: El sistema deberá mostrar el historial de movimientos de la billetera virtual.

Gestión de Panel de Usuario (Mis Actividades)

RF-37: El sistema deberá mostrar las subastas en las que el usuario haya participado.

RF-38: El sistema deberá indicar si el usuario ganó, perdió o continúa participando en cada subasta.

RF-39: El sistema deberá mostrar las publicaciones creadas por el vendedor.

RF-40: El sistema deberá visualizar el estado y la recaudación de cada publicación.

Gestión Administrativa

RF-41: El sistema deberá permitir al administrador gestionar las categorías de productos.

RF-42: El sistema deberá permitir al administrador suspender o habilitar usuarios de la plataforma.

RF-43: El sistema deberá permitir al administrador moderar publicaciones que incumplan las políticas del sistema.

RF-44: El sistema deberá consultar el historial de subastas y transacciones realizadas.

Gestión de Procesos Automáticos y Auditoría

RF-45: El sistema deberá finalizar automáticamente las subastas cuyo tiempo haya expirado mediante un proceso en segundo plano.

RF-46: El sistema deberá adjudicar la subasta al postor con la oferta más alta y registrar la venta cuando exista un ganador.

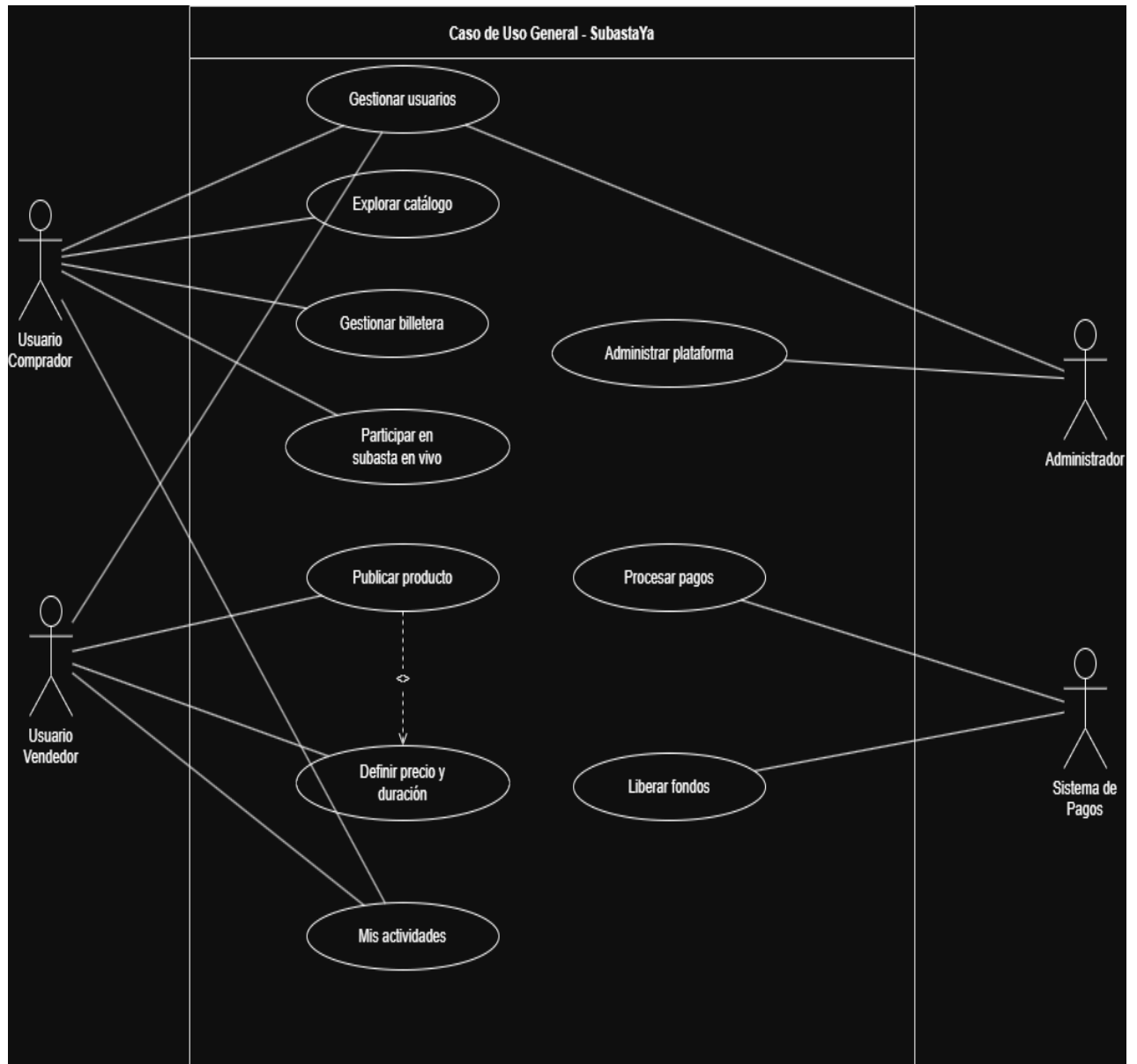
RF-47: El sistema deberá marcar una subasta como desierta cuando finalice sin ofertas registradas.

RF-48: El sistema deberá registrar en una bitácora de auditoría los cambios de estado de las subastas, las extensiones por anti-sniping, las acreditaciones de saldo y los intentos de puja rechazados.

Casos de uso.

Definición de casos de uso: los casos de uso, descripciones detalladas de las interacciones entre actores y el sistema.

En este informe se dará un caso de uso general.



Requerimientos no funcionales (RNF).

Definición:

Seguridad

RNF-01: El sistema deberá cifrar las credenciales y la información sensible de los usuarios.

RNF-02: El sistema deberá garantizar que las operaciones de la billetera virtual sean transaccionales e íntegras.

RNF-03: El sistema deberá impedir la utilización de fondos no disponibles durante el proceso de puja.

RNF-04: El sistema deberá registrar un historial auditable de las operaciones económicas realizadas.

Rendimiento

RNF-05: El sistema deberá reflejar las nuevas ofertas en tiempo real para los usuarios conectados.

RNF-06: El sistema deberá actualizar el contador regresivo con una latencia mínima durante las subastas activas.

RNF-07: El sistema deberá soportar múltiples usuarios ofertando simultáneamente sin pérdida de consistencia.

Usabilidad

RNF-08: El sistema deberá presentar una interfaz web intuitiva y de fácil navegación.

RNF-09: El sistema deberá ser responsivo y adaptarse correctamente a dispositivos móviles, tabletas y computadoras.

RNF-10: El sistema deberá mantener una presentación visual uniforme en todos los módulos de la aplicación.

Disponibilidad y Confiabilidad

RNF-11: El sistema deberá mantener la disponibilidad del servicio durante las subastas en curso.

RNF-12: El sistema deberá recuperar de forma consistente el estado de las subastas ante fallos inesperados.

RNF-13: El sistema deberá asegurar que una subasta tenga un único ganador válido al momento de su cierre.

Mantenibilidad

RNF-14: El sistema deberá desarrollarse con una arquitectura modular que facilite su mantenimiento y evolución.

RNF-15: El sistema deberá documentar los servicios y componentes principales para facilitar futuras modificaciones.

Modelo de dominio.

Definición. El modelo de dominio es una representación conceptual del problema que buscamos resolver, describe las entidades principales, sus atributos y las relaciones entre ellas, lo más precisa posible pues mientras sea más precisa también será mas sencillo tratar las soluciones.

Si bien guarda similitud con los actores del sistema, la principal diferencia es que los actores son entidades externas , mientras que las entidades del diagram del modelo de dominio son internos.

Características:

Abstracción del negocio: refleja cómo funciona el sistema desde el punto de vista lógico, no técnico.

Independencia tecnológica: no depende del lenguaje de programación ni del motor de base de datos.

Relaciones claras: muestra asociaciones, agregaciones y composiciones entre entidades.

Base para el diseño UML: se traduce luego en diagramas de clases y objetos.

Con esto en mente logramos identificar las siguientes entidades:

Entidades del Modelo de Dominio

Entidad	Descripción	Relaciones
Usuario	Representa a compradores, vendedores y administradores registrados en el sistema.	Se asocia con Billetera, Subasta y Puja.
Billetera	Contiene el saldo total, el saldo retenido y el saldo disponible de cada usuario.	Se relaciona con Usuario y Transacción.
Categoría	Clasifica las subastas según el tipo de producto (Tecnología, Coleccionables, Indumentaria, Vehículos, etc.).	Se relaciona con Subasta.

Subasta	Representa la publicación de un producto con su período de inicio, cierre y estado.	Se asocia con Producto, Categoría, Puja y Usuario (vendedor).
Producto	Elemento que el vendedor publica para ser subastado.	Pertenece a una Subasta.
Puja	Oferta realizada por un comprador durante una subasta.	Se asocia con Usuario y Subasta.
Transacción	Registra los movimientos financieros de la billetera, como depósitos, retenciones, liberaciones y liquidaciones.	Se vincula con Billetera.
Ledger / Auditoría	Almacena de forma inmutable los eventos críticos y las operaciones relevantes del sistema.	Se relaciona con Subasta y Transacción.

Diagrama entidad relación (DER).

Definición:

El **diagrama entidad-relación** es una representación gráfica del modelo de datos de un sistema. Su objetivo es mostrar **cómo se relacionan las entidades entre sí** y qué atributos posee cada una. Se utiliza para diseñar la estructura lógica de la base de datos antes de implementarla.

Propósito:

- Visualizar la estructura de la base de datos de forma clara y organizada.
- Detectar redundancias o inconsistencias antes de crear las tablas.
- Servir como guía para generar el modelo físico en SQL o mediante el ORM.

Esta misma no es sugerida por la cátedra en la pautas del TP.

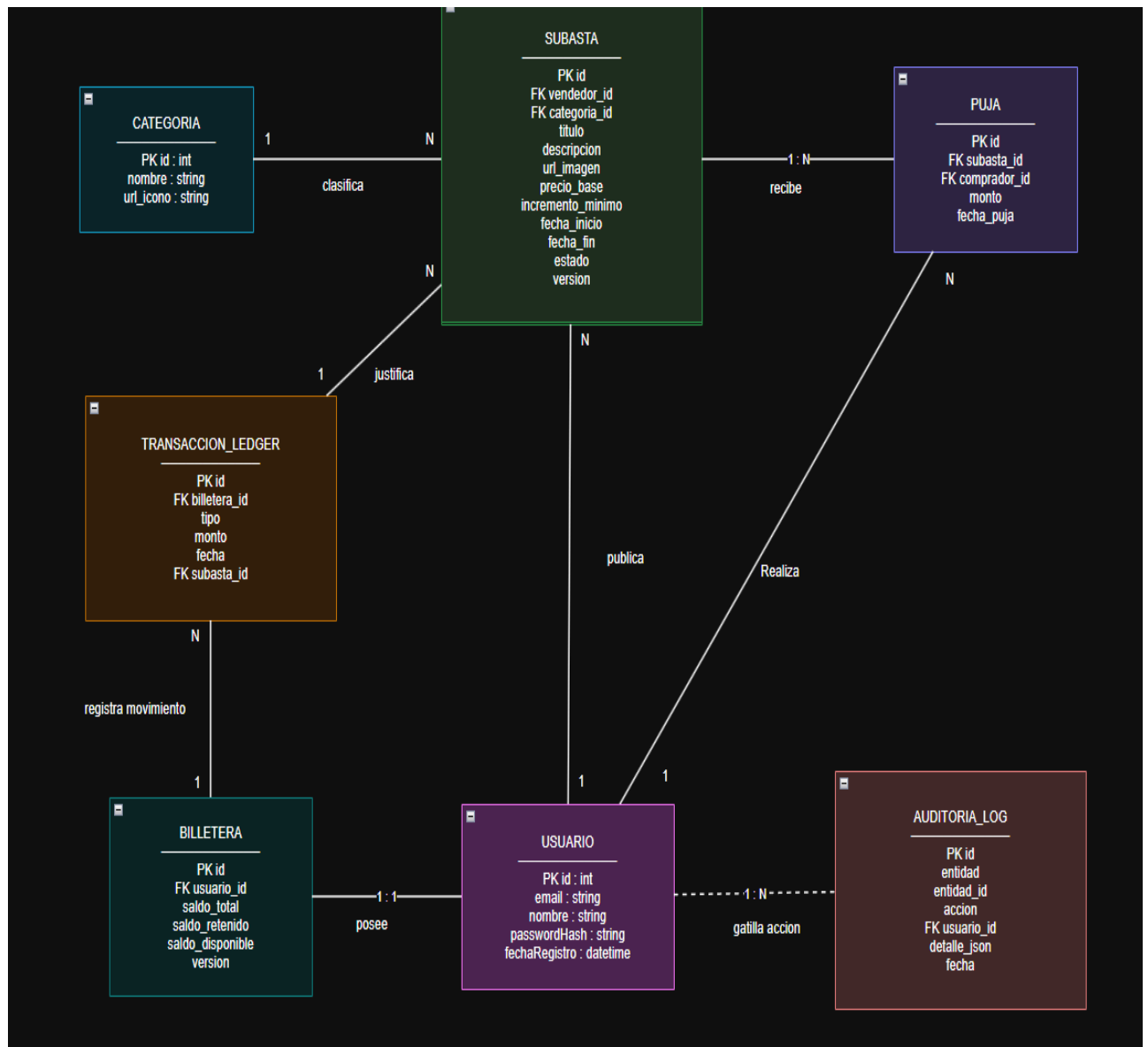
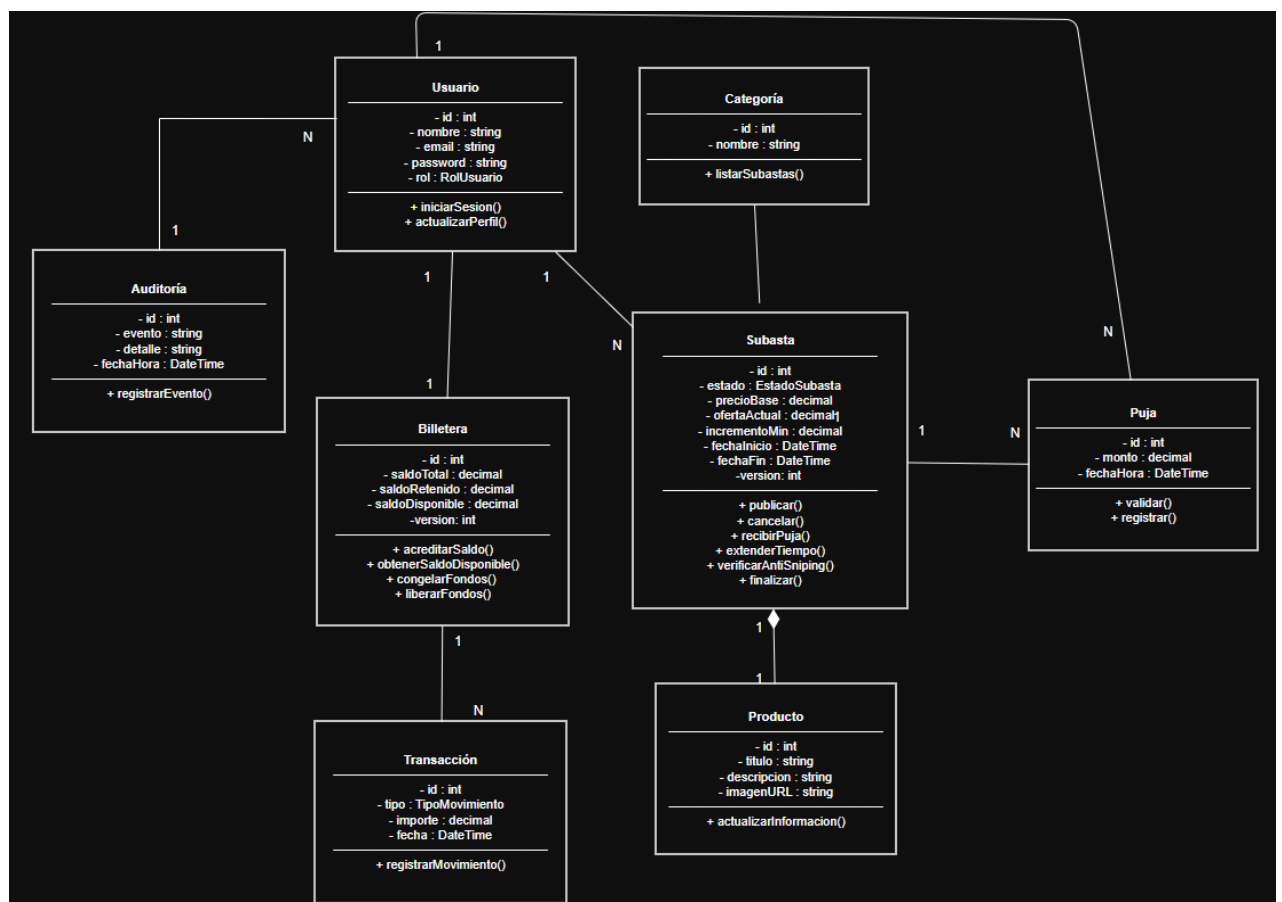


Diagrama de clases:

Definición: el diagrama de clases o UML es un diagrama que nos permite representar de manera precisa antes de llevarlo a programación tanto las entidades u objeto de nuestro sistema, en el UML podemos ver de manera visual cómo se comportará nuestro sistema, de esta manera poder hacer modificaciones o mejoras antes de programarlo.

Además nos sirve para mostrarle al cliente de manera visual, como va ser el sistema, y su comportamiento de esta manera, el cliente puede hacer cambios si es lo que quiere.



Arquitectura del sistem.

La arquitectura del sistema “SubastaYa” se basa en un **modelo por capas** que separa las responsabilidades en distintos niveles para garantizar mantenibilidad y escalabilidad.

Las capas principales son:

- **Presentación:** vistas web, formularios de login, panel de usuario, listado de subastas.
- **Aplicación:** controladores, servicios de subasta, billetera y auditoría.
- **Dominio:** clases Usuario, Subasta, Puja, Billetera, TransaccionLedger, AuditoriaLog.

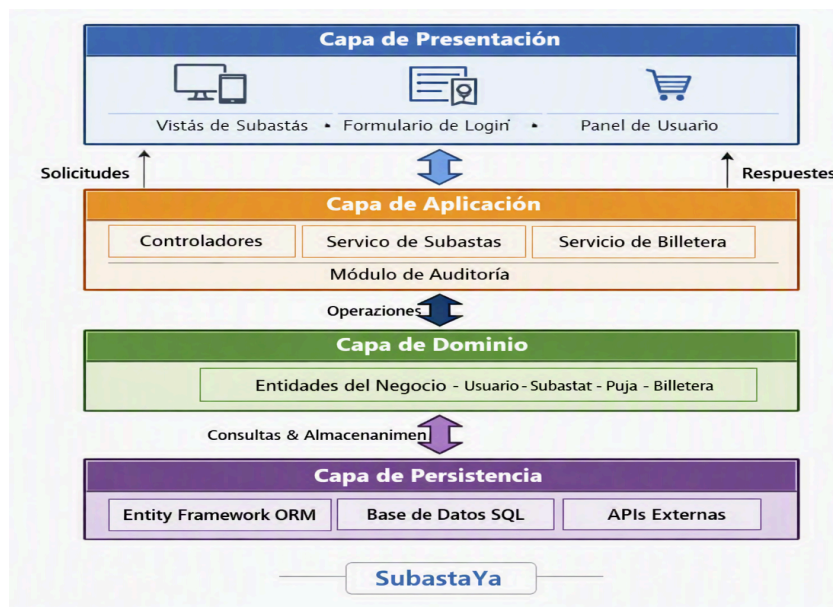
- **Persistencia:** DbContext, migraciones, repositorios y configuración del ORM.

Esta arquitectura permite una comunicación clara entre componentes y facilita la evolución del sistema sin afectar su núcleo funcional.

Flujo de comunicación entre capas

Una breve descripción de cómo se comunican las capas.

- La capa de presentación **envía solicitudes** a la capa de aplicación.
- La capa de aplicación **usa las entidades del dominio** para procesar la lógica.
- La capa de persistencia **guarda y recupera datos** del sistema.



Desarrollo del backend.

Ahora que tenemos diseñado las bases de nuestro proyecto como, identificación de actores, RF, RNF, modelo de dominio y la arquitectura de nuestro sistema estamos en condiciones para el desarrollo de la parte de **backend**.

Definición general

El backend es la parte del sistema encargada de procesar la lógica del negocio, gestionar las operaciones internas y comunicarse con la base de datos.

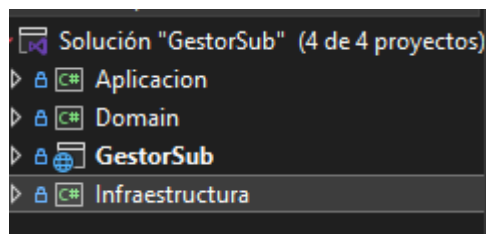
En *SubastaYa*, el backend será responsable de manejar las subastas, las pujas, la billetera virtual y la auditoría de transacciones.

Lenguaje y tecnologías utilizadas

- Lenguaje: C#, java, html.
- Framework: ASP.NET Core 8
- ORM: Entity Framework Core (enfoque Code First)
- Base de datos: SQL Server
- Patrón arquitectónico: Modelo por capas (Presentación, Aplicación, Dominio, Persistencia)

Estructura del backend

1. Capa de Dominio: define las entidades (Usuario, Subasta, Puja, Billetera, TransaccionLedger, AuditoriaLog) y sus reglas de negocio.
2. Capa de Aplicación: implementa los servicios (ServicioDeSubastas, ServicioDeBilletera, ServicioDeAuditoria) y coordina la lógica entre las capas.
3. Capa de Persistencia: maneja el acceso a datos mediante el ORM y define el DbContext.
4. API REST: expone los endpoints para el frontend y otros servicios externos.



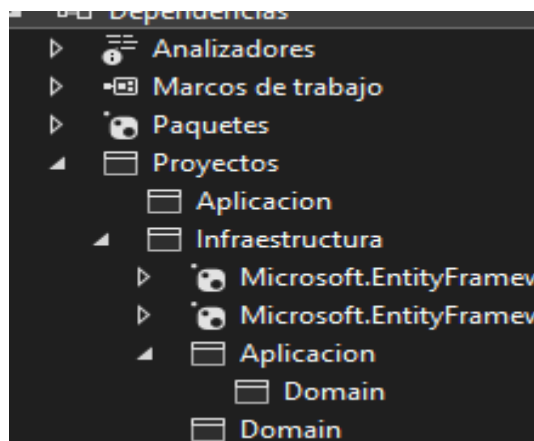
GestorEs es la capa de presentación. Pero para lograr la estructura de capas debemos trabajar con las dependencias:

Aplicación: depende de **Dominio**.

Infraestructura depende de **Aplicación**.

Presentacion (GestorSub) depende de **infraestructura**.

En la siguiente imagen se ve como se logro dicha **dependencia**:



Flujo de funcionamiento

1. El frontend envía una solicitud HTTP (por ejemplo, una nueva puja).
2. La API REST recibe la solicitud y la pasa al servicio correspondiente.
3. El servicio de aplicación valida las reglas del negocio (saldo suficiente, incremento mínimo, tiempo restante).
4. El dominio ejecuta la lógica y actualiza las entidades.
5. La persistencia guarda los cambios en la base de datos mediante Entity Framework.
6. Se devuelve una respuesta al frontend con el resultado (éxito, error o actualización en tiempo real).

Herramientas complementarias

- Swagger: para documentar y probar los endpoints.
- AutoMapper: para mapear DTOs y entidades.
- SignalR: para comunicación en tiempo real durante las subastas.
- JWT: para autenticación segura de usuarios.

Desarrollo del Backend:

Para el desarrollo del Backend, y el buen administrado del mismo usaremos la metodología ágil **Scrum**.

Metodología de desarrollo – Scrum

Scrum es una metodología ágil que organiza el desarrollo en iteraciones cortas llamadas Sprints. Cada Sprint produce un incremento funcional del software y permite dividir el trabajo entre los integrantes del equipo.

Conceptos principales

- **Product Backlog:** lista priorizada de todos los requerimientos del proyecto.
- **Sprint:** período de trabajo donde se implementan los RF seleccionados.
- **Sprint Backlog:** conjunto de tareas que el equipo realizará durante ese Sprint.
- **Incremento:** resultado funcional obtenido al finalizar la semana.

De esta manera se el equipo se puede organizar de manera más eficiente, pero, como solo dos personas, en el Sprint las tareas se separan por gestión, por lo que los requisitos en el product backlog se realizan por gestión. Al igual que el Sprint se realizará por gestión, dichas gestiones están separadas también en **backend** y **frontend**

Product Backlog del Backend

Prioridad	Gestión	RF	Story Points
1	Gestión de Usuarios	RF-01 al RF-04	8
2	Creación de Subastas	RF-11 al RF-17	13

3	Sala de Subasta en Vivo	RF-18 al RF-29	21
4	Billetera Virtual	RF-30 al RF-36	13
5	Procesos Automáticos	RF-45 al RF-48	8

Product Backlog del Frontend

Prioridad	Gestión	RF	Story Points
1	Catálogo y Exploración	RF-05 al RF-10	8
2	Panel de Usuario	RF-37 al RF-40	5
3	Gestión Administrativa	RF-41 al RF-44	5

¿Qué son esos puntos?

No representan horas. Son una medida relativa de complejidad:

- 5 = baja
- 8 = media
- 13 = alta
- 21 = muy alta

Sprint Backend

Día	Gestión	Encargado
Lunes	Gestión de Usuarios	
Martes	Creación de Subastas	
Miércoles	Billetera Virtual	
Jueves	Sala de Subasta en Vivo	
Viernes	Procesos Automáticos y Testing	

Sprint – Frontend

Día	Gestión	Encargado
Lunes	Maquetado y Login	
Martes	Catálogo y Filtros	
Miércoles	Publicación de Subastas	

Jueves	Panel de Usuario	
Viernes	Panel Administrativo e Integración	